



.NET/Server

Global Exception Handling and Error Management in .NET

Building safer, cleaner, and more resilient applications



“Why Error Management Matters”



Applications fail in unexpected ways



Users need clear and safe error responses



Developers need useful logs for debugging



Repeated try-catch blocks make code harder to maintain



Global exception handling centralizes error behavior

“Good error management improves **reliability**, **security**, and **maintainability**.”

What Is an Exception?

An exception is an unexpected runtime event that interrupts the normal flow of an application.



Null reference



Invalid input



Database
connection failure



Unauthorized
access



External API
timeout

In .NET, exceptions are represented as objects derived from **System.Exception**.

Local Try-Catch vs Global Exception Handling

Local Try-Catch



Used for specific recovery



Useful when the method can handle the problem



Can become repetitive if overused

Global Exception Handling



Centralizes unhandled errors



Creates consistent API responses



Improves logging and monitoring



Keeps controllers and services cleaner

Use **local try-catch** only when you can actually recover or add meaningful context.



The Problem With Repeated Try-Catch

BAD EXAMPLE

```
try
{
    var user = service.GetUser(id);
    return Ok(user);
}
catch (Exception ex)
{
    logger.LogError(ex, "Error");
    return StatusCode(500);
}
```

PROBLEMS

- Duplicated error logic
- Inconsistent responses
- Controllers become noisy
- Easy to forget logging
- Harder to maintain

Global Exception Handling Concept

Global exception handling catches unhandled exceptions in one central place before the application sends a response.



The goal is not to hide errors, but to handle them consistently.



ASP.NET Core Middleware Pipeline

ASP.NET Core processes each HTTP request through a middleware pipeline. Exception handling middleware can catch errors from later components.



The exception handler should be registered early in the pipeline.



Using `ExceptionHandler`

```
app.UseExceptionHandler("/error");

app.Map("/error", (HttpContext context) =>
{
    return Results.Problem(
        title: "An unexpected error occurred.",
        statusCode: 500
    );
});
```

- Redirects unhandled exceptions to a central endpoint
- Prevents exposing internal exception details
- Returns a consistent error response
- Useful for production environments



ProblemDetails: Standard Error Format

ProblemDetails provides a standard structure for HTTP API error responses.

```
{  
  "type": "https://example.com/errors/internal-server-  
error",  
  "title": "An unexpected error occurred.",  
  "status": 500,  
  "detail": "Please contact support if the problem  
continues.",  
  "traceId": "00-abcd1234"  
}
```

- type
- title
- status
- detail
- traceId



Custom Global Exception Handler

A custom handler maps application exceptions to clear HTTP responses.

NotFoundException	→	404 Not Found	
ValidationException	→	400 Bad Request	
UnauthorizedAccessException	→	401 Unauthorized	
ConflictException	→	409 Conflict	
Unexpected Exception	→	500 Internal Server Error	

- Different exception types should produce different HTTP responses.
- Provides meaningful context to API consumers.
- Streamlines API consumer error handling.



Custom Exceptions

```
public class NotFoundException : Exception
{
    public NotFoundException(string message)
        : base(message)
    {
    }
}

// Example usage:
throw new NotFoundException("User was not found.");
```



- Custom exceptions make business errors explicit and easier to handle globally.
- Custom exceptions should represent meaningful application errors.

Logging Exceptions Correctly

- ✓ Log the full exception object
- ✓ Include useful context
- ✓ Avoid logging sensitive data
- ✓ Use structured logging
- ✓ Include correlation or trace IDs
- ✓ Send logs to monitoring tools



```
// Example usage:  
logger.LogError(ex, "Error processing  
user {UserId}", userId);
```



*Logs are for **developers**
and **operations teams**, not
for end users.*

Do Not Expose Internal Errors

Production error responses should not reveal sensitive implementation details.

Do Not Expose:

- ❌ Stack traces
- ❌ Database names
- ❌ SQL queries
- ❌ Server paths
- ❌ Connection strings
- ❌ Internal class names
- ❌ Secret keys or tokens



"An unexpected error occurred. Please try again later."



Recommended Strategy in .NET

- ① Use global exception handling middleware
- ② Return ProblemDetails for APIs
- ③ Map custom exceptions to HTTP status codes
- ④ Log detailed errors internally
- ⑤ Show safe messages to users
- ⑥ Use validation for expected input problems
- ⑦ Add trace IDs for troubleshooting
- ⑧ Test error scenarios

**“Handle errors globally,
log them internally, and
respond safely.”**



Key Takeaways

- Exceptions interrupt normal application flow
- Repeated try-catch blocks create messy code
- Global exception handling centralizes error behavior
- ProblemDetails creates consistent API responses
- Logs should contain details, but users should see safe messages
- Good error management improves security and reliability

**How should your
application respond
when something
goes wrong?**
